Unit 4 Seminar Practical Activity: Structural Design Patterns –Decorator Pattern

The objective of this exercise was to demonstrate how the Decorator Pattern can be used to add new functionality to an existing object dynamically without modifying its original structure. As part of the Unit 4 seminar requirements, I applied the pattern through a simple coffee-shop application using a Coffee interface, a SimpleCoffee class, and decorators such as Milk, Sugar, and Whip.

Python Implementation

1. **Abstract Coffee Class:**

   The Coffee class defines the common cost() and description() behaviour that every coffee component must implement.

```python
from abc import ABC, abstractmethod


class Coffee(ABC):

    @abstractmethod
    def cost(self) -> float:
        """
        Define the common cost behaviour.
        Every coffee type must implement
        its own version of this method.
        """
        raise NotImplementedError

    @abstractmethod
    def description(self) -> str:
        """
        Define the common description behaviour.
        """
        raise NotImplementedError
```

2. **Concrete Component-Simple Coffee:**

The SimpleCoffee class is the concrete component that provides the basic coffee implementation, including its original cost and description.

```
class SimpleCoffee(Coffee):

    def cost(self) -> float:
        """
        Return the cost of a basic coffee.
        """
        return 2.00

    def description(self) -> str:
        """
        Return the description of a basic coffee.
        """
        return "Simple coffee"
```

3. **Base Decorator – CoffeeDecorator:**

   CoffeeDecorator defines the common structure for all decorators. It stores a Coffee object and forwards the cost() and description() calls to the wrapped object.

```
class CoffeeDecorator(Coffee):

    def __init__(self, coffee: Coffee):
        """
        Store the coffee object that will be decorated.
        """
        self.coffee = coffee

    def cost(self) -> float:
        return self.coffee.cost()

    def description(self) -> str:
        return self.coffee.description()
```

4. **Concrete Decorators**

The Milk, Sugar, and Whip classes are concrete decorators. Each decorator extends the existing coffee object by adding its own cost and description.

```
# Each decorator adds a new feature to the existing coffee object.

class Milk(CoffeeDecorator):

    def cost(self) -> float:
        """
        Add the cost of milk to the existing coffee.
        """
        return self.coffee.cost() + 0.50

    def description(self) -> str:
        return self.coffee.description() + " + milk"


class Sugar(CoffeeDecorator):

    def cost(self) -> float:
        """
        Add the cost of sugar to the existing coffee.
        """
        return self.coffee.cost() + 0.20

    def description(self) -> str:
        return self.coffee.description() + " + sugar"


class Whip(CoffeeDecorator):

    def cost(self) -> float:
        """
        Add the cost of whipped cream to the existing coffee.
        """
        return self.coffee.cost() + 0.70

    def description(self) -> str:
        return self.coffee.description() + " + whip"
```

5. Applying the Decorators

The main program first creates a SimpleCoffee object. The decorators are then applied one after another, with each decorator wrapping the previous coffee object.

```
if __name__ == "__main__":

    # Start with a simple coffee.
    coffee = SimpleCoffee()

    # Add toppings dynamically using decorators.
    coffee_with_milk = Milk(coffee)
    coffee_with_milk_and_sugar = Sugar(coffee_with_milk)
    deluxe_coffee = Whip(coffee_with_milk_and_sugar)
```

6. **Storing and Displaying the Coffee Objects**

The different coffee configurations are stored in a list. The program then uses the same description() and cost() methods to display the final description and total cost of each coffee.

```
coffees = [
        coffee,
        coffee_with_milk,
        coffee_with_milk_and_sugar,
        deluxe_coffee
    ]

    # Display the description and total cost of each coffee.
    for item in coffees:
        print(f"{item.description()}: ${item.cost():.2f}")
```

7. **Program Output:**

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Code> python.exe .\unit4_decorator.py
Simple coffee + milk + sugar: $2.70
Simple coffee + milk + sugar + whip: $3.40
```